Introduction to Computers and Programming in C

Module V

Pointers

Pointers

Pointer declaration:

A pointer is a variable that contains the memory location of another variable. We start by specifying the type of data stored in the location identified by the pointer. The asterisk tells the compiler that we are creating a pointer variable. Finally we give the name of the variable.

The syntax is as shown below.

```
type *variable_name
```

Example:

```
int *ptr;
float *ptr1;
```

Address Operator

Once we declare a pointer variable, we must point it to something we can do this by assigning to the pointer the address of the variable you want to point as in the following example:

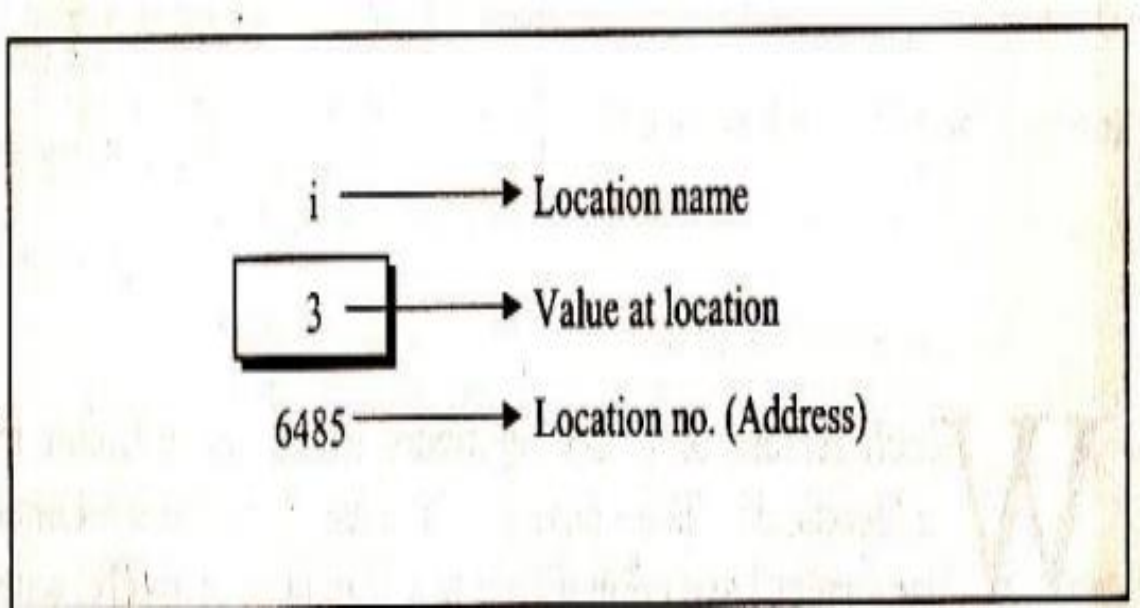
```
ptr=&num;
```

Another Example

```
int i=3;
```

- Reserve space in memory to hold the integer value
- Associate the name i with this memory location.
- Store the value 3 at this location

Pointers



Source: Let Us C, Yashvant Kanetkar, BPB Publications

Pointers

```
main( )
{
    int i = 3 ;

    printf ( "\nAddress of i = %u", &i ) ;
    printf ( "\nValue of i = %u", i ) ;
}
```

The output of the above program would be:

```
Address of i = 6485
Value of i = 3
```

Source: Let Us C, Yashvant Kanetkar, BPB Publications

Indirection Operator

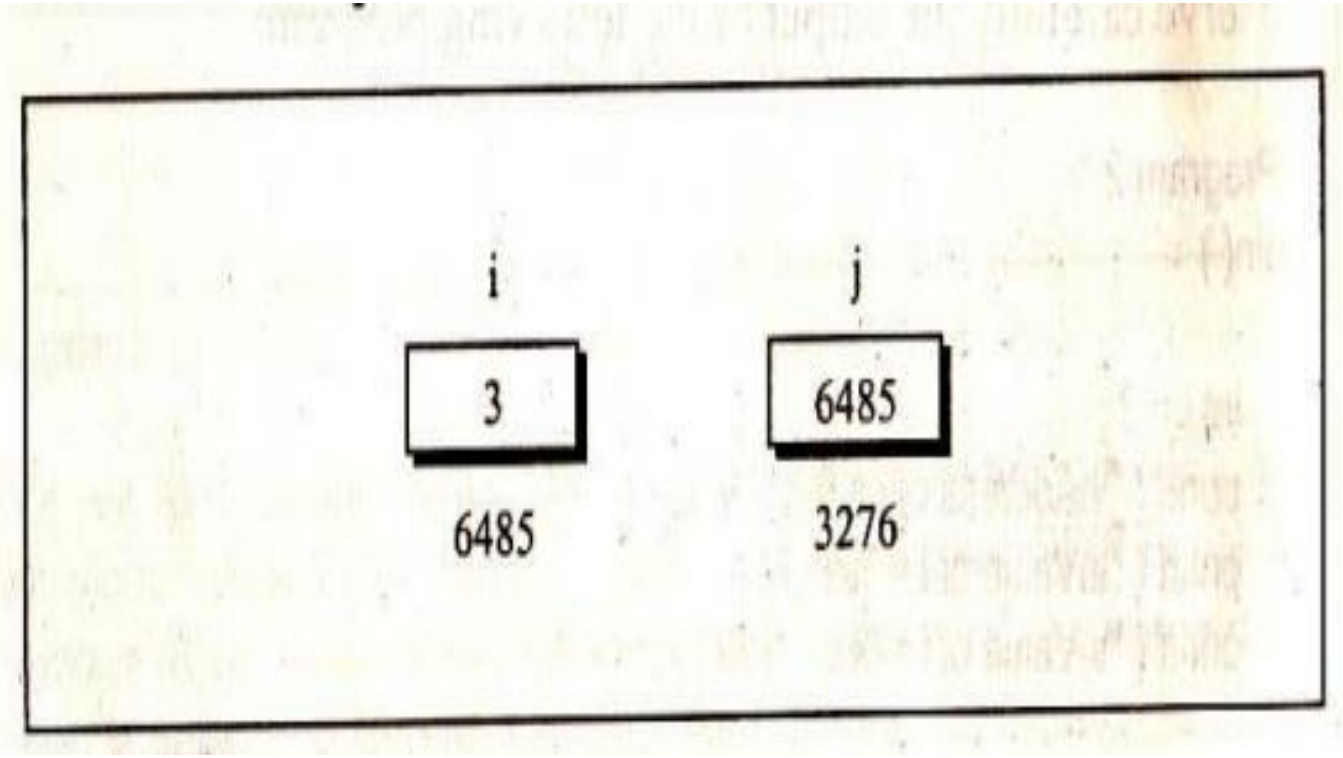
Value at address pointer (*)-Returns the value stored at a particular address (indirection operator).

```
main( )
{
    int i = 3 ;
    printf ( "\nAddress of i = %u", &i ) ;
    printf ( "\nValue of i = %d", i ) ;
    printf ( "\nValue of i = %d", *( &i ) ) ;
}
```

Printing the value of *(&i) is same as printing the value of i.

Source: Let Us C, Yashvant Kanetkar, BPB Publications

Pointers



Source: Let Us C, Yashvant Kanetkar, BPB Publications

Pointers

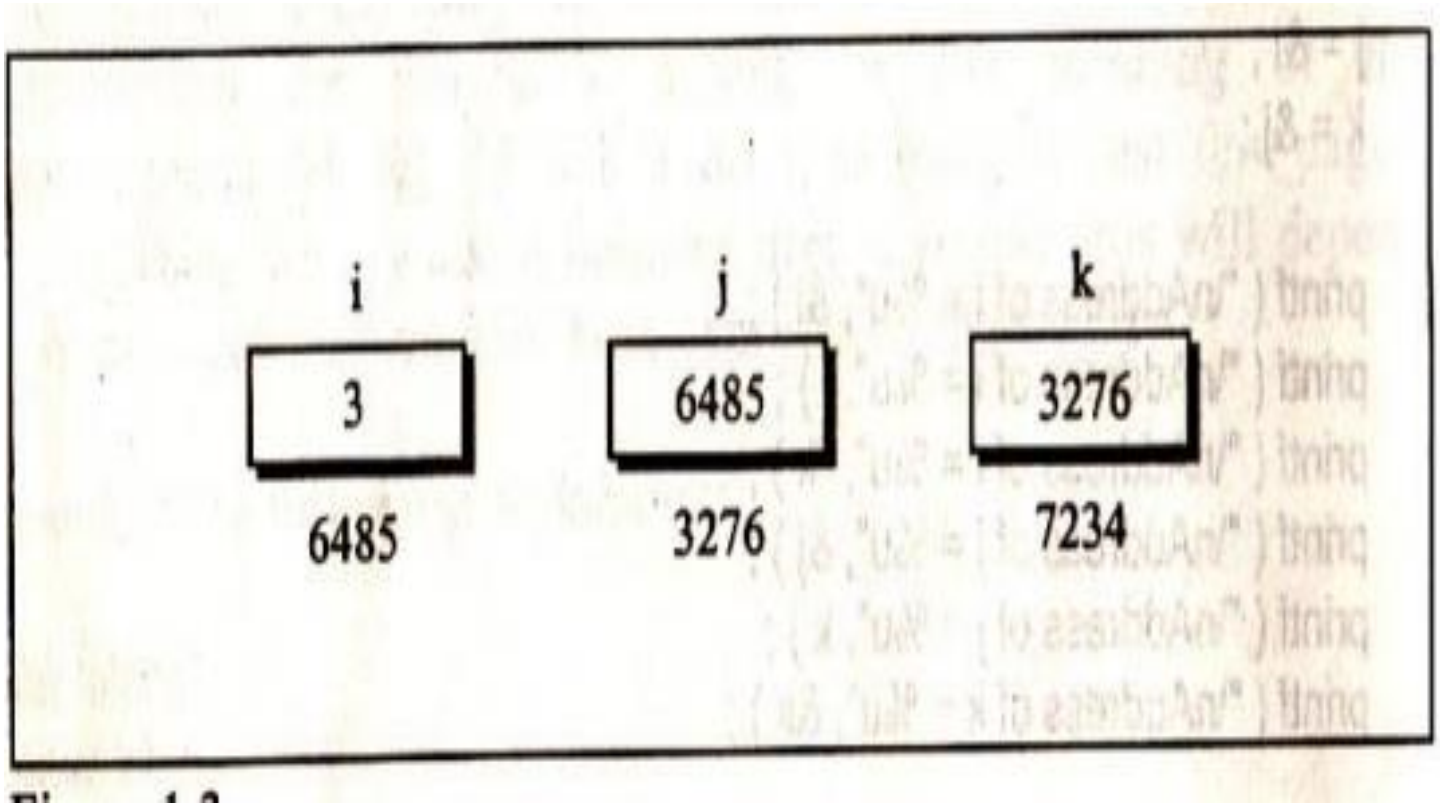
- Pointers are variables that contain addresses and since addresses are always whole numbers, pointers always contain whole numbers.

Pointers

❖float *s

- The declaration float *s does not mean that s is going to contain a floating-point value. It means that s is going to contain the address of a floating-point value.

Pointers



Source: Let Us C, Yashvant Kanetkar, BPB Publications

Pointers

```
main( )
{
        int i = 3 ;
        int *j ;
        int **k ;

        j = &i ;
        k = &j ;

        printf ( "\nAddress of i = %u", &i ) ;
        printf ( "\nAddress of i = %u", j ) ;
        printf ( "\nAddress of i = %u", *k ) ;
        printf ( "\nAddress of j = %u", &j ) ;
        printf ( "\nAddress of j = %u", k ) ;
        printf ( "\nAddress of k = %u", &k ) ;

        printf ( "\n\nValue of j   = %u", j ) ;
        printf ( "\nValue of k   = %u", k ) ;
        printf ( "\nValue of i   = %d", i ) ;
        printf ( "\nValue of i   = %d", *( &i ) ) ;
        printf ( "\nValue of i   = %d", *j ) ;
        printf ( "\nValue of i   = %d", **k ) ;
}
```

Source: Let Us C, Yashvant Kanetkar, BPB Publications

Jargon of Pointers

```
int a=35;
int *b;
b=&a;
```

- b contains address of an int
- Value at address contained in b is an int
- b is an int pointer
- b points to an int
- b is a pointer which points in direction of an int.

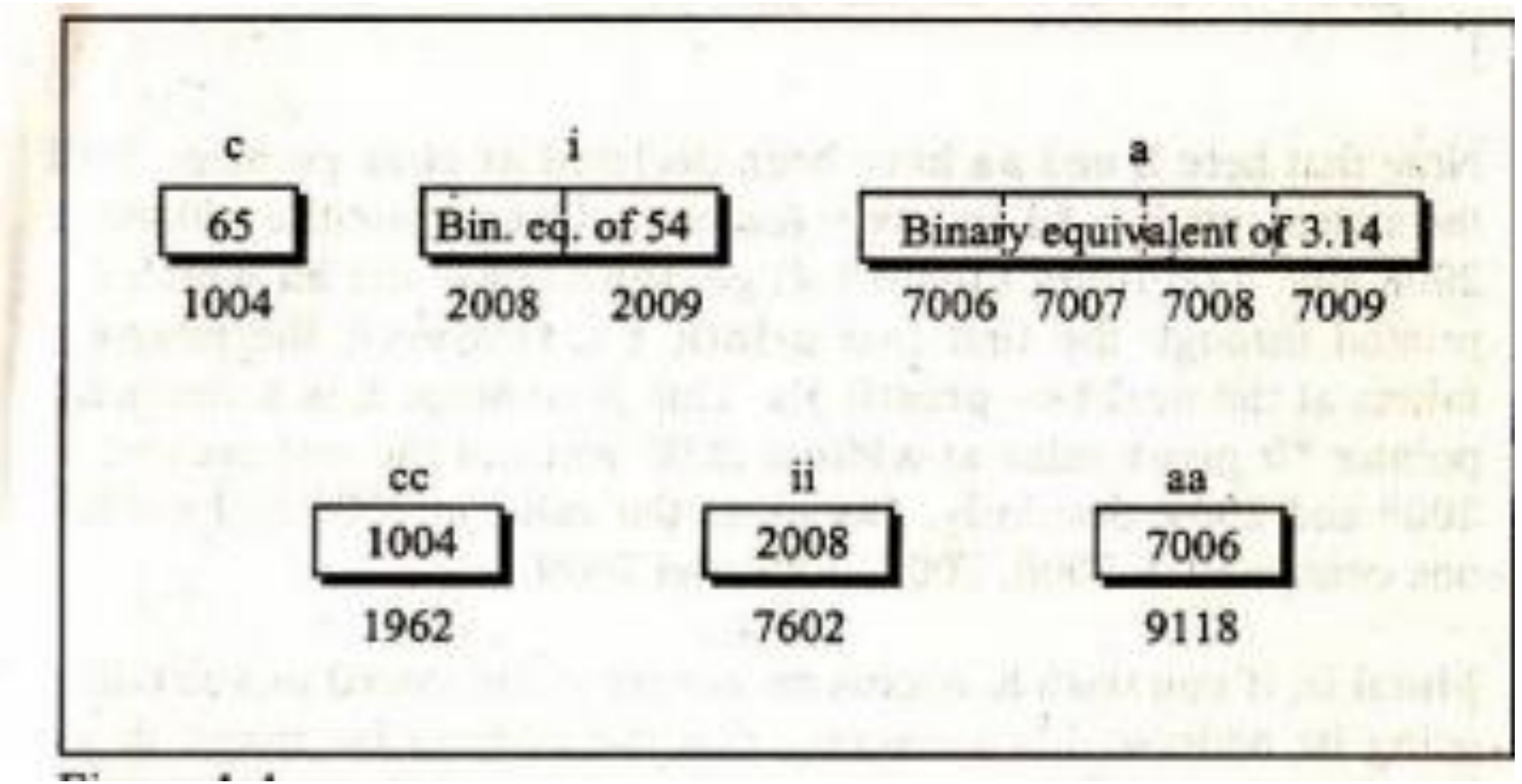
```
/* Program to display the contents of the variable their address using pointer variable*/

#include<stdio.h>
void main()
{
    int num, *intptr;
    float x, *floptr;
    char ch, *cptr;
    num=123;
    x=12.34;
    ch='a';
    intptr=&num;
    cptr=&ch;
    floptr=&x;
    printf("Num %d stored at address %u\n",*intptr,intptr);
    printf("Value %f stored at address %u\n",*floptr,floptr);
    printf("Character %c stored at address %u\n",*cptr,cptr);
}
```

char,int and float pointer

```
main()
{
char c,*cc;
int i, *ii;
float a,*aa;
c=‘A’ //ASCII value stored in c
i=54;
a=3.14;
cc=&c;ii=&i;aa=&a;
Printf(“address contained in cc = %u", cc);
Printf(“address contained in ii = %u", ii);
Print(“address contained in aa = %u", aa);
printf(“value of c=%c”, *cc);
}
```

char,int and float pointer



Source: Let Us C, Yashvant Kanetkar, BPB Publications

Pointer expressions & pointer arithmetic

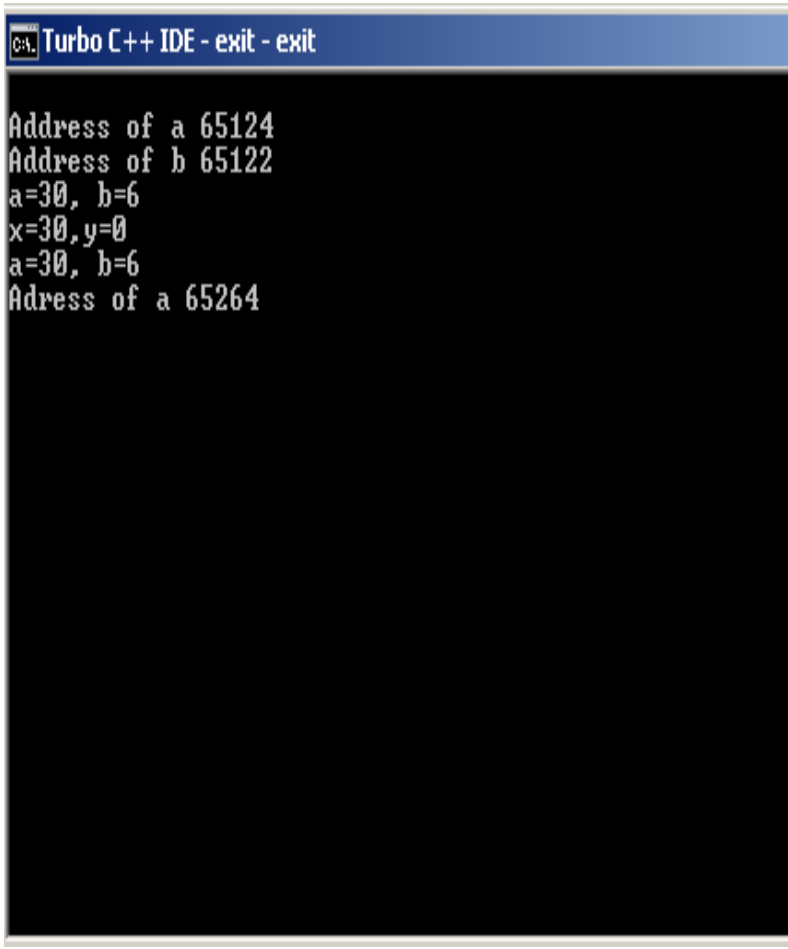
Like other variables pointer variables can be used in expressions.

For example if p1 and p2 are properly declared and initialized pointers, then the following statements are valid.

```
y = *p1 * *p2;
sum = sum + *p1;
z = 5 - *p2/*p1;
*p2 = *p2 + 10;
```

we can also compare pointers by using relational operators the expressions such as
p1 >p2 , p1==p2 and p1!=p2 are allowed.

```
/*Program to illustrate the pointer expression and pointer arithmetic*/
#include<stdio.h>
#include<conio.h>
void main()
{
    int *ptr1,*ptr2;
    int a,b,x,y,z;
    clrscr();
    a=30;b=6;
    ptr1=&a;
    ptr2=&b;
    x=*ptr1+ *ptr2-6; /*        30+6-6 =30         */
    y=6* (- *ptr1) / *ptr2 + 30; /*  6*(-30)/6+30=0 */
    printf("\nAddress of a %u",ptr1);
    printf("\nAddress of b %u",ptr2);
    printf("\na=%d, b=%d",a,b);
    printf("\nx=%d,y=%d",x,y);
    ptr1=ptr1 + 70;
    printf("\na=%d, b=%d",a,b);
    printf("\nAddress of a %u",ptr1);
    getch();
}
```



Passing address to functions

Arguments are generally passed to functions in one of the two ways

- Sending the value of the arguments
- Sending the addresses of the arguments

Sending the values of the arguments

- ‘value’ of each **actual argument** in the calling function is copied into corresponding **formal argument** of the called function.
- Changes made to the formal arguments in the called function have no effect on the values of actual arguments in the calling function.

Call by value

```
main()
{
int a =10, b=20;
Swap(a,b);
Printf(“\na=%d”,a);
Printf(“\n b=%d”,b);
}
Swap(int x, int y)
{
int t;
t=x;
x=y;y=t;
printf(“\nx=%d“, x);
printf (“\ny = %d",y );
}
```

Call by reference

- The address of actual arguments in the calling function are copied into formal arguments of the called function.
- Using formal arguments in the called function changes can be made in the actual arguments of the calling function.

Call by Reference

```
main( )
{
    int a = 10 ;
    int b = 20 ;

    swapr ( &a, &b ) ;
    printf ( "\na = %d", a ) ;
    printf ( "\nb = %d", b ) ;
}

swapr ( int *x, int *y )
{
    int t ;

    t = *x ;
    *x = *y ;
    *y = t ;
}
```

Source: Let Us C, Yashvant Kanetkar, BPB Publications

Call by reference

- Using call by reference function can return more than one value at a time.

```
main()
{int radius; float area, perimeter;
Printf(“Enter the radius of a circle”);
scanf(“%d”,&radius);
Areaperi(radius,&area,&perimeter);
printf(%f %f,area,perimeter);
}
Areaperi (int r,float *a,float *p)
{
*a= 3.14*r*r;
*p= 2*3.14*r;
}
```

Functions Returning Pointers

```
main()
{
Int *p;
Int *fun(); // prototype declaration
p=fun();
printf(“\n%u”,p);
printf(“\n%d”,*p);
}
Int *fun() //function definition
{
int i=20;
return(&i);
}
```

*ptr++ and ++*ptr

- *ptr++ increments the pointer not the value pointed by it.
- ++*ptr increments the value being pointed to by ptr.

Void pointer

```
main()
{
int a=10, *j;
void *k;
j=k=&a;
j++;k++;
printf(“\n %u %u”,j,k);
}
```

Passing Array Elements

- Array elements can be passed to a function by calling the function

(a) By value

(b) By reference

Passing Array ELements

Call by value

```
main( )
{
    int i ;
    int marks[ ] = { 55, 65, 75, 56, 78, 78, 90 } ;

    for ( i = 0 ; i <= 6 ; i++ )
        display ( marks[i] ) ;
}

display ( int m )
{
    printf ( "\n%d", m ) ;
}
```

Call by reference

```
main( )
{
    int i ;
    int marks[ ] = { 55, 65, 75, 56, 78, 78, 90 } ;

    for ( i = 0 ; i <= 6 ; i++ )
        disp ( &marks[i] ) ;
}
disp ( int *n )
{
    printf ( "\n%d", *n ) ;
}
```

Source: Let Us C, Yashvant Kanetkar, BPB Publications

Pointers and Arrays

```
main()
{
int i=3, *x;
float j=1.5, *y;
char k= ‘c’,*z;
printf(“%d%f%c”,i,j,k);
x=&i,y=&j,z=&k;
printf(%u%u%u”,x,y,z);
x++;y++;z++;
printf(%u%u%u”,x,y,z);
}
```

Pointers and Arrays

- Do not attempt the following operations on Pointers

Addition of two pointers

Multiplying pointer with a number

Dividing a pointer with a number

```
int i=4,*j,*k;
j=&i;
j=j+1,j=j-5;            //Allowed
```

Accessing Array Elements

```
main()
{
int num[]={24,34,12,44,56,17};
int i=0;
while(i<=5)
{
printf(“\n address is %u”,&num[i]);
printf(“element=%d”,num[i]);
i++;
}
}
```

Accessing Array Elements- using pointer

```
main()
{
int num[]={24,34,12,44,56,17};
int i=0, *j;
j=&num[0];
while(i<=5)
{
printf("\n address is %u",&num[i]);
printf("element=%d",*j);
i++;
j++;                    // increment pointer to the next location
}
}
```

Passing an Entire Array to a function

```
main()
{
int num[]={24,34,12,44,56,17};
display(&num[0],6);          // or display(num,6);
}
display( int *j, int n)
{ int i=1;
While(i<=n)
{
printf("element=%d",*j);
i++;
j++;  // increment pointer to point the next location
}
```

Accessing array element in different ways

```
main( )
{
    int num[ ] = { 24, 34, 12, 44, 56, 17 } ;
    int i = 0 ;

    while ( i <= 5 )
    {
        printf ( "\naddress = %u    " , &num[i] ) ;
        printf ( "element = %d  ", num[i] ) ;
        printf ( "%d  ", *( num + i ) ) ;
        printf ( "%d  ", *( i + num ) ) ;
        printf ( "%d", i[num] ) ;
        i++ ;
    }
}
```

The output of the program would look like this:

```
address = 4001    element = 24  24  24  24
address = 4003    element = 34  34  34  34
address = 4005    element = 12  12  12  12
address = 4007    element = 44  44  44  44
address = 4009    element = 56  56  56  56
address = 4011    element = 17  17  17  17
```

Source: Let Us C, Yashvant Kanetkar, BPB Publications

2-D Array

```
main( )
{
    int stud[5][2] = {
                        { 1234, 56 },
                        { 1212, 33 },
                        { 1434, 80 },
                        { 1312, 78 },
                        { 1203, 75 }
                    };
    int i, j ;

    for ( i = 0 ; i <= 4 ; i++ )
    {
        printf ( "\n" ) ;
        for ( j = 0 ; j <= 1 ; j++ )
            printf ( "%d  ", stud[i][j] ) ;
    }
}
```

2-D Array using Pointer

```
main( )
{
      int stud[5][2] = {
                              { 1234, 56 },
                              { 1212, 33 },
                              { 1434, 80 },
                              { 1312, 78 },
                              { 1203, 75 }
                        } ;
      int i, j ;

      for ( i = 0 ; i <= 4 ; i++ )
      {
            printf ( "\n" ) ;
            for ( j = 0 ; j <= 1 ; j++ )
                  printf ( "%d  ", *( *( stud + i ) + j ) ) ;
      }
}
```

And here is the output...

```
1234   56
1212   33
1434   80
1312   78
1203   75
```

Source: Let Us C, Yashvant Kanetkar, BPB Publications

Pointer to an array

- As we have pointer to int,float and char we can have pointer to an array
- For example int (*q)[4] means that q is a pointer to an array of 4 integers

Pointer to an array

```
main( )
{
    int a[ ][4] =   {
                        5, 7, 5, 9,
                        4, 6, 3, 1,
                        2, 9, 0, 6
                    };
    int *p ;
    int ( *q )[4] ;

    p = ( int* ) a ;
    q = a ;

    printf ( "\n%u %u", p, q ) ;
    p++ ;
    q++ ;
    printf ( "\n%u %u", p, q ) ;
}
```

And here is the output...

```
65500 65500
65502 65508
```

Source: Let Us C, Yashvant Kanetkar, BPB Publications

Passing 2-D array to a function

```
main( )
{
    int a[3][4] = {
                        1, 2, 3, 4,
                        5, 6, 7, 8,
                        9, 0, 1, 6
                  };

    clrscr( ) ;
    display ( a, 3, 4 ) ;
    show ( a, 3, 4 ) ;
    print ( a, 3, 4 ) ;
}

display ( int *q, int row, int col )
{
    int i, j ;

    for ( i = 0 ; i < row ; i++ )
    {
        for ( j = 0 ; j < col ; j++ )
            printf ( "%d ", * ( q + i * col + j ) ) ;

        printf ( "\n" ) ;
    }
```

Source: Let Us C, Yashvant Kanetkar, BPB Publications

Passing 2-D array to a function

```
show ( int ( *q )[4], int row, int col )
{
    int i, j ;
    int *p ;

    for ( i = 0 ; i < row ; i++ )
    {
        p = q + i ;
        for ( j = 0 ; j < col ; j++ )
            printf ( "%d ", * ( p + j ) ) ;

        printf ( "\n" ) ;
    }
    printf ( "\n" ) ;
}

print ( int q[ ][4], int row, int col )
{
    int i, j ;

    for ( i = 0 ; i < row ; i++ )
    {
        for ( j = 0 ; j < col ; j++ )
            printf ( "%d ", q[i][j] ) ;

        printf ( "\n" ) ;
    }
    printf ( "\n" ) ;
}
```

And here is the output...

```
1 2 3 4
5 6 7 8
9 0 1 6
```

Source: Let Us C, Yashvant Kanetkar, BPB Publications

Returning Array from Function

- A pointer to an integer
- A pointer to the zeroth 1-D array
- A pointer to the 2-D array

Array of Pointers

- A pointer variable always contains an address, an array of pointers would be nothing but collection of addresses.

```
Main()
{
Static int a[]={0,1,2,34};
Static int *p[]={a,a+1,a+2,a+3,a+4};
printf("\n%u%u%d",p,*p,*(*p));
}
```

Dynamic Menory Allocation

- If we want to allocate memory at the time of excecution then it can be done using standard library functions calloc() and malloc() and are known as dynamic memory allocation functions.
- In C, malloc is a memory allocation function that dynamically allocates a large block of memory with a specified size. It's useful when you don't know the amount of memory needed during compile time.
- The calloc() function in C is used to allocate multiple blocks of memory that are the same size. It's a dynamic memory allocation function that sets each block to zero. calloc() stands for "contiguous allocation".

Example

```
int main()
{
   int n,avg,i,*p,sum=0;
   printf("\nEnter the number of students");
   scanf("%d",&n);
   p=(int *)malloc(n*2);
   if(p==NULL)
   {
      printf("Memory allocation unsuccessful");
      exit(0);
   }
   for(i=0;i<n;i++)
   {
      scanf("%d",(p+i));
         }
   for(i=0;i<n;i++)
{
sum=sum+*(p+i);
}
avg=sum/n;
printf("Avarage marks=%d",avg);
   return 0;
}
```

Example problem

```
main( )
{
    int a[ ] = { 10, 20, 30, 40, 50 } ;
    int j ;
    for ( j = 0 ; j < 5 ; j++ )
    {
        printf ( "\n%d", *a ) ;
        a++ ;
    }
}
```

Source: Let Us C, Yashvant Kanetkar, BPB Publications

References

- Programming in ANSI C, E. Balagurusamy. McGrawHill
- Let Us C, Yashvant Kanetkar, BPB Publications
- Programming in C, Reema Thareja, Oxford University Press
- https://www.dotnettricks.com

Thank You